

Khawaja Fareed Graduate College

Department of Computer Science



FINAL YEAR PROJECT

AI-Powered Web-Based Smart Attendance System

Contactless attendance using Face Recognition & GPS Geofencing

Submitted by **Ayesha Yousaf**

Session **2023 – 2025**

Program **Computer Science**

Institution **Khawaja Fareed Graduate College**

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement & Objectives
4. Scope & Limitations
5. Technology Stack
6. System Architecture
7. Functionality (User Roles & Modules)
8. Core Features Explained
9. Implementation Details
10. Database Design
11. Application Walkthrough (Screenshots)
12. Security
13. Deployment
14. Testing & Verification
15. Conclusion & Future Enhancements

Note. The working system is deployed under the institutional brand shown in the screenshots. The project itself is a general-purpose *AI-Powered Web-Based Smart Attendance System* and can be re-branded for any institution.

1. Abstract

Manual attendance in classrooms is slow, error-prone, and easy to abuse through proxy ("buddy") marking. This project presents a **web-based smart attendance system** that verifies **two facts** before recording a student present: **identity** (the face belongs to the logged-in student) and **presence** (the student is physically inside the classroom/campus radius). A teacher opens a time-bounded, GPS-restricted attendance session; a student logs in on their phone, confirms their location, and captures their face; the server runs an eleven-point validation pipeline — session state, time window, GPS accuracy and radius, face detection, and one-to-one face matching — before saving the record.

The system is built entirely on a **zero-cost technology stack** (React, FastAPI, and a free-tier PostgreSQL database) and uses a lightweight on-device/on-server AI model for face recognition, making it deployable without any paid biometric hardware or services. It is also an installable **Progressive Web App (PWA)**, giving students a native-app-like experience on mobile phones.

2. Introduction

2.1 Background

Attendance is a fundamental record in every academic institution, but traditional methods — roll-call, paper registers, or signature sheets — waste class time, are difficult to audit, and are commonly defeated by students marking attendance on behalf of absent friends. Card- or fingerprint-based biometric systems solve some of these problems but require dedicated hardware at every location, which is expensive to install and maintain.

2.2 Proposed Solution

This project replaces dedicated hardware with the devices students already own. Using a normal smartphone browser, the system combines two independent, hard-to-fake signals:

- **Face recognition** confirms *who* is marking attendance — the captured face must match the face registered to that exact logged-in account, so one student cannot mark for another.
- **GPS geofencing** confirms *where* the student is — attendance is accepted only when the device is inside a radius the teacher sets around the classroom.

Both checks are enforced on the server, and attendance can only be marked while a teacher has an active session open, giving teachers full control over when and where students may check in.

3. Problem Statement & Objectives

3.1 Problem Statement

To design and implement a low-cost, secure, web-based attendance system that eliminates proxy attendance and manual record-keeping by automatically verifying a student's **identity** and **physical presence** at the time of marking.

3.2 Objectives

- Provide role-based access for **Administrators, Teachers, and Students**.
- Allow teachers to create **GPS-restricted, time-bounded attendance sessions**.
- Verify student identity through **face recognition** tied to the logged-in account.
- Verify student presence through **GPS radius and accuracy checks**.
- Automatically compute attendance status (**present / late / absent**) from session timing.
- Support **manual attendance** and an **approve/reject** review flow for doubtful cases.
- Generate **reports and CSV exports** and role-specific **dashboards**.
- Use only **free and open-source** tools and hosting.
- Deliver an installable, mobile-friendly **Progressive Web App**.

4. Scope & Limitations

4.1 In Scope

- Three user roles with role-based routing and permissions.
- Admin management of users, classes, subjects, and teacher assignments.
- Session-based attendance with one active window per class.
- Face registration and one-to-one face verification.
- GPS geofence with distance and accuracy validation.
- Manual marking, approvals, reports (CSV), and dashboards.
- Activity logging of key actions.
- Installable PWA with offline app-shell caching.

4.2 Limitations

- GPS can be spoofed on rooted/jailbroken devices; for higher assurance it can be paired with QR codes or campus Wi-Fi verification (listed as future work).
- Face recognition accuracy depends on lighting and camera quality.
- Advanced anti-spoofing / liveness detection and mask-aware recognition are future enhancements.

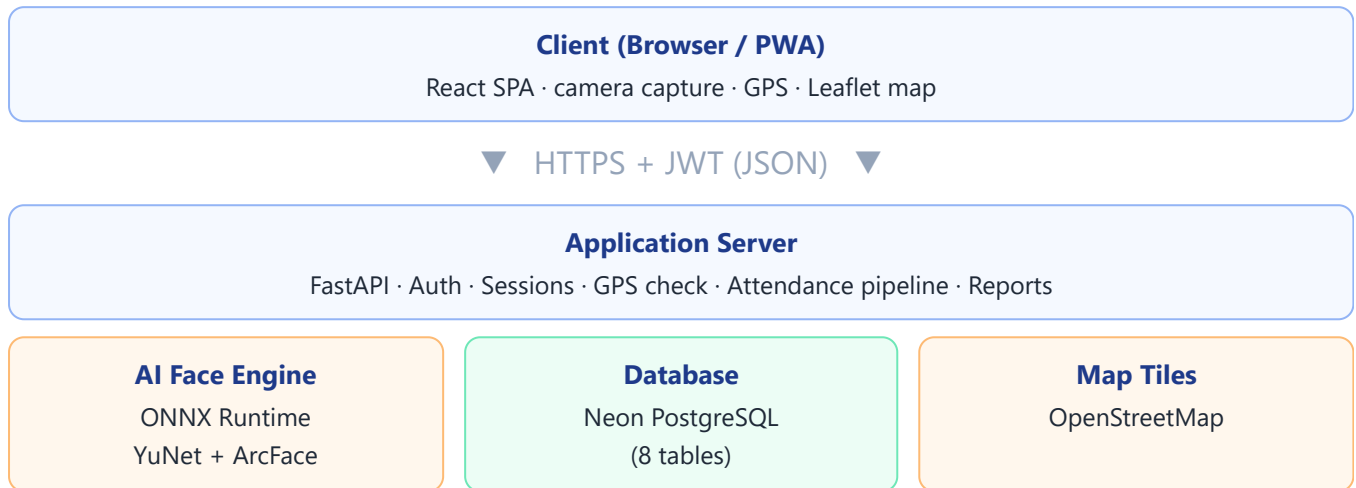
5. Technology Stack

Every component was chosen to be free, open-source, and deployable on a no-cost hosting tier.

Layer	Technology	Why it was chosen
Frontend	React 18 + Vite, React Router, Axios	Fast, component-based SPA with a modern build tool and client-side routing.
UI / Design	Custom CSS design system, Recharts-style inline SVG charts	Consistent, lightweight, dependency-free interface with dashboards and charts.
Maps	Leaflet + OpenStreetMap	Free, open map tiles for picking and displaying the attendance location.
Camera guidance	MediaPipe Face Landmarker (in-browser)	Guided, auto-capturing face capture with a live pose indicator.
Backend	FastAPI (Python) + Uvicorn	High-performance, typed API framework with automatic documentation.
ORM / Validation	SQLAlchemy 2.0, Pydantic v2	Safe database access and strict request/response validation.
Authentication	JWT (PyJWT) + bcrypt (passlib)	Stateless tokens and industry-standard password hashing.
Face recognition (AI)	ONNX Runtime — YuNet detector + ArcFace (MobileFaceNet) embeddings	Accurate, lightweight face embeddings that run on CPU within a 512 MB server.
Database	Neon PostgreSQL (free tier)	Managed, serverless PostgreSQL with zero cost for the project scale.
Hosting	Render (backend), Vercel (frontend)	Free-tier deployment with HTTPS, required for camera & GPS.
App delivery	Progressive Web App (vite-plugin-pwa)	Installable on phones with splash screen and offline app shell.

6. System Architecture

The application follows a classic **three-tier client-server architecture**. A React single-page app runs in the browser, communicates with a FastAPI backend over HTTPS using JSON and JWTs, and the backend persists all data in a managed PostgreSQL database.



6.1 Attendance Data Flow

1. Teacher opens a session (class, subject, time window, location, radius).
2. Student opens *Mark Attendance*; the app fetches the active session.
3. Browser captures GPS coordinates and accuracy; the server checks the geofence.
4. Browser captures the face; the server computes an embedding and compares it to the student's registered embeddings.
5. The server runs all validation checks, computes present/late status from the time, and saves the record. The student sees a success or rejection message.

7. Functionality (User Roles & Modules)

The system defines three roles. After login, each user is routed to a role-specific dashboard.

7.1 Administrator

- Manage **students and teachers** (create, edit, reset password, activate/deactivate).
- Manage **classes and subjects**; assign teachers to class–subject pairs.
- View **all attendance** records, generate **reports**, and export **CSV**.
- View **dashboard analytics** and the **activity log**.

7.2 Teacher

- View assigned classes/subjects and a personal dashboard.
- **Create attendance sessions** with location, radius, date, and time window.
- **Mark attendance manually** (with a required reason) and **close** sessions.
- **Approve or reject** pending/doubtful records and export class reports.

7.3 Student

- **Register their face** through a guided, auto-capturing camera flow.
- **Mark attendance** using face recognition + GPS while a session is open.
- View their **attendance percentage** and history, and update their profile.

8. Core Features Explained

8.1 Session-Based Attendance

Students cannot mark attendance at arbitrary times. A teacher must first open a session for a class and subject with a **start time**, **end time**, and optional **late cut-off**. To keep the "which session am I marking?" question unambiguous, the system enforces **one active attendance window per class** — overlapping sessions for the same class are rejected — and expired sessions are **automatically closed**.

8.2 Attendance Status Logic

Time of marking	Result
Before start time	Rejected — session not started
Between start and end time	Present
Between end time and late cut-off	Late
After late cut-off	Rejected — session closed

8.3 GPS Geofencing

The browser reports the student's latitude, longitude, and accuracy. The server computes the great-circle distance to the session location using the **Haversine formula**. Attendance is accepted only if the reading is accurate enough (accuracy within a configurable limit) and the distance is within the allowed radius. If the student is clearly outside the radius, the message says so explicitly rather than blaming accuracy.

8.4 Face Recognition (One-to-One Verification)

During registration, several photos of the student are captured; each face is detected, aligned, and converted into a 512-dimension numerical **embedding** that is stored against the student's account. At marking time, the captured face is embedded the same way and compared (using **cosine similarity**) **only against the logged-in student's own embeddings**. This design directly prevents proxy attendance: even a perfectly recognised face is rejected if it does not belong to the account marking attendance.

8.5 The Validation Pipeline

Before any record is saved, the server verifies, in order:

1. Student is authenticated.
2. Account is active.
3. Student belongs to the session's class.
4. Session is open.
5. Current time is inside the window.
6. GPS coordinates were provided.
7. GPS accuracy is acceptable.
8. Student is inside the radius.
9. A face is detected.
10. The face matches the logged-in student.
11. No duplicate record for this session.

If any check fails, nothing is saved and a clear reason is returned.

8.6 Manual Attendance & Approvals

Teachers can mark a student manually (with a mandatory reason). Borderline face matches are saved as **pending** for review; a teacher can then **approve** (which resolves the record to present/late based on when it was marked) or **reject** it (which records the student as absent).

9. Implementation Details

Authentication & Roles

Passwords are hashed with **bcrypt**. On login, the server issues a signed **JWT** containing the user id and role; the frontend stores it and attaches it to every request. Every protected endpoint is guarded by a role dependency, and login accepts an **email (case-insensitive) or registration number**.

Face Engine

Face detection uses **YuNet** (via OpenCV), which returns the face box and five facial landmarks. The face is aligned to a canonical template and passed to an **ArcFace MobileFaceNet** model running on **ONNX Runtime**, producing an L2-normalised 512-D embedding. Because it runs on CPU and the models total ~14 MB, the whole engine fits within a free 512 MB server — a deliberate design decision after an initial TensorFlow-based prototype proved too heavy for free hosting.

Verification. On the project's test images, the same person scored a cosine similarity of **0.66–0.93** while a different person scored ≈ 0.0 , giving a wide, safe separation for the decision threshold.

Time-Zone Correctness

Session times are entered as local wall-clock times, but the server runs in UTC. All time-window logic is therefore computed through a single configurable timezone offset, so sessions show the correct state (upcoming / present / late / closed) regardless of where the server is hosted.

Reports & Dashboards

Attendance can be filtered by date range, class, subject, student, and status, and exported as **CSV**. Each role sees a tailored dashboard with counts, an attendance-trend chart, and a recent-activity list; students additionally see an attendance-percentage ring.

Progressive Web App

A service worker caches the application shell for fast, offline-capable loading, and a web manifest with icons makes the app installable. A branded splash screen (Android and iOS) and a custom pull-to-refresh give it a native feel.

10. Database Design

The PostgreSQL schema contains eight related tables:

Table	Purpose & key fields
users	Admins, teachers, students in one table — name, email, password hash, role, registration number, class, semester, department, active/face-registered flags.
classes	Class / program groups — name, program, semester.
subjects	Subjects within a class — name, code, class reference.
teacher_classes	Assignment of a teacher to a class + subject.
face_embeddings	Per-student 512-D face vectors, active flag.
attendance_sessions	Teacher-created session — class, subject, date, times, latitude/longitude, radius, status.
attendance_records	One row per student per session — status, method, approval, confidence, distance, GPS accuracy, reason, timestamps (unique per session+student).
activity_logs	Audit trail of key actions (login, face registration, attendance, etc.).

11. Application Walkthrough

The following screenshots demonstrate the running system on desktop and mobile.

The login screen is a split-screen desktop layout. The left panel has a blue background and contains the following elements:

- IUB Attendance System** logo and title.
- Attendance, verified by face & location.** (Large white text)
- A description: "A secure, contactless attendance system for students and teachers at the Islamia University of Bahawalpur."
- Three features listed with icons:
 - Face-verified identity
 - GPS-restricted sessions
 - Real-time attendance reports
- Islamia University of Bahawalpur (at the bottom)

The right panel is white and contains the login form:

- Welcome back** (Section header)
- Sign in to continue to your dashboard. (Subtext)
- Email or registration number** (Label)
- (Input field)
- Password** (Label)
- (Input field)
- Sign in** (Blue button)
- [Forgot your password? Ask an administrator to reset it.](#) (Link)

Fig 11.1 — Login screen (split-screen desktop layout). Users sign in with email or registration number.

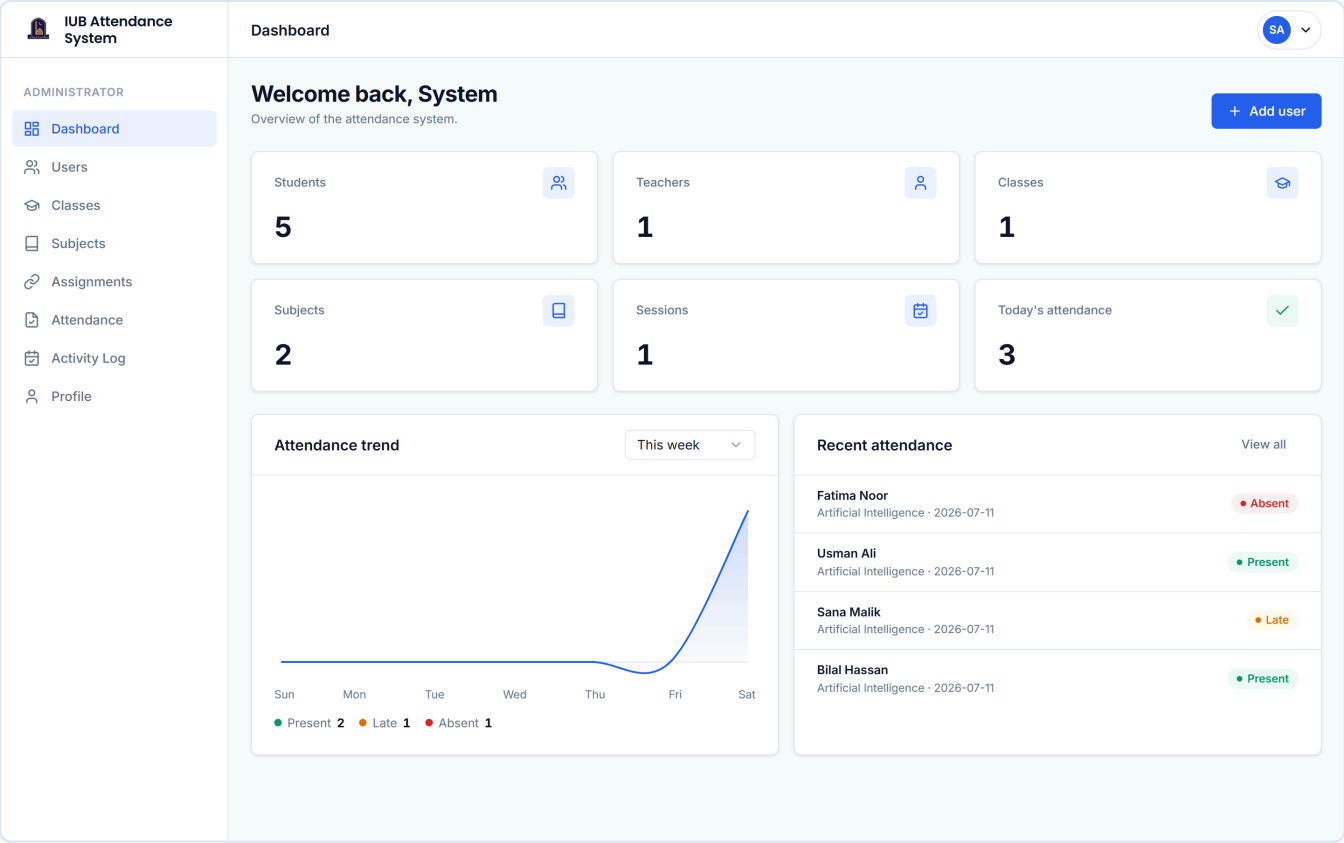


Fig 11.2 — Administrator dashboard: live counts, attendance-trend chart, and recent activity.

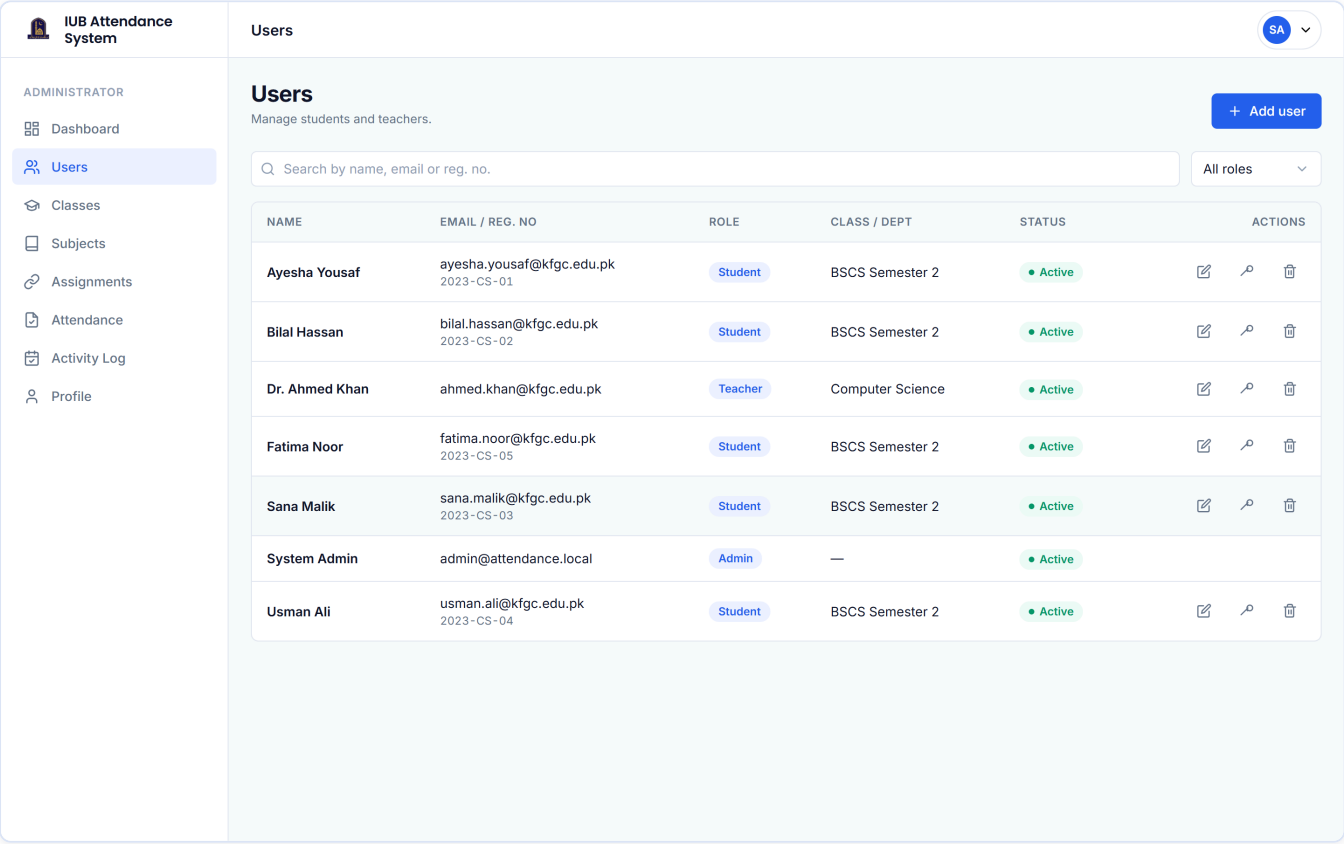


Fig 11.3 — User management: search, filter, add/edit students and teachers, reset passwords.

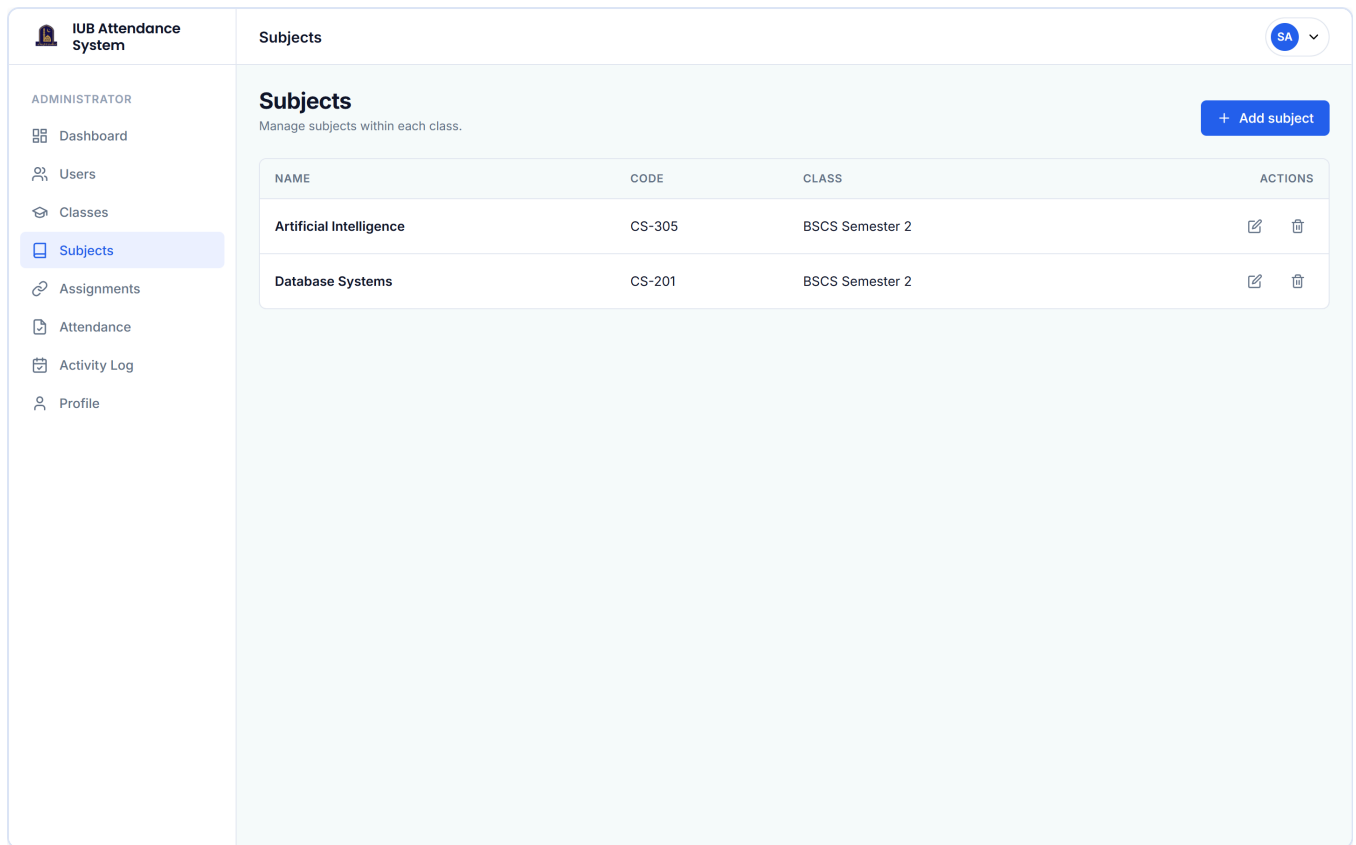


Fig 11.4 — Subject management within a class.

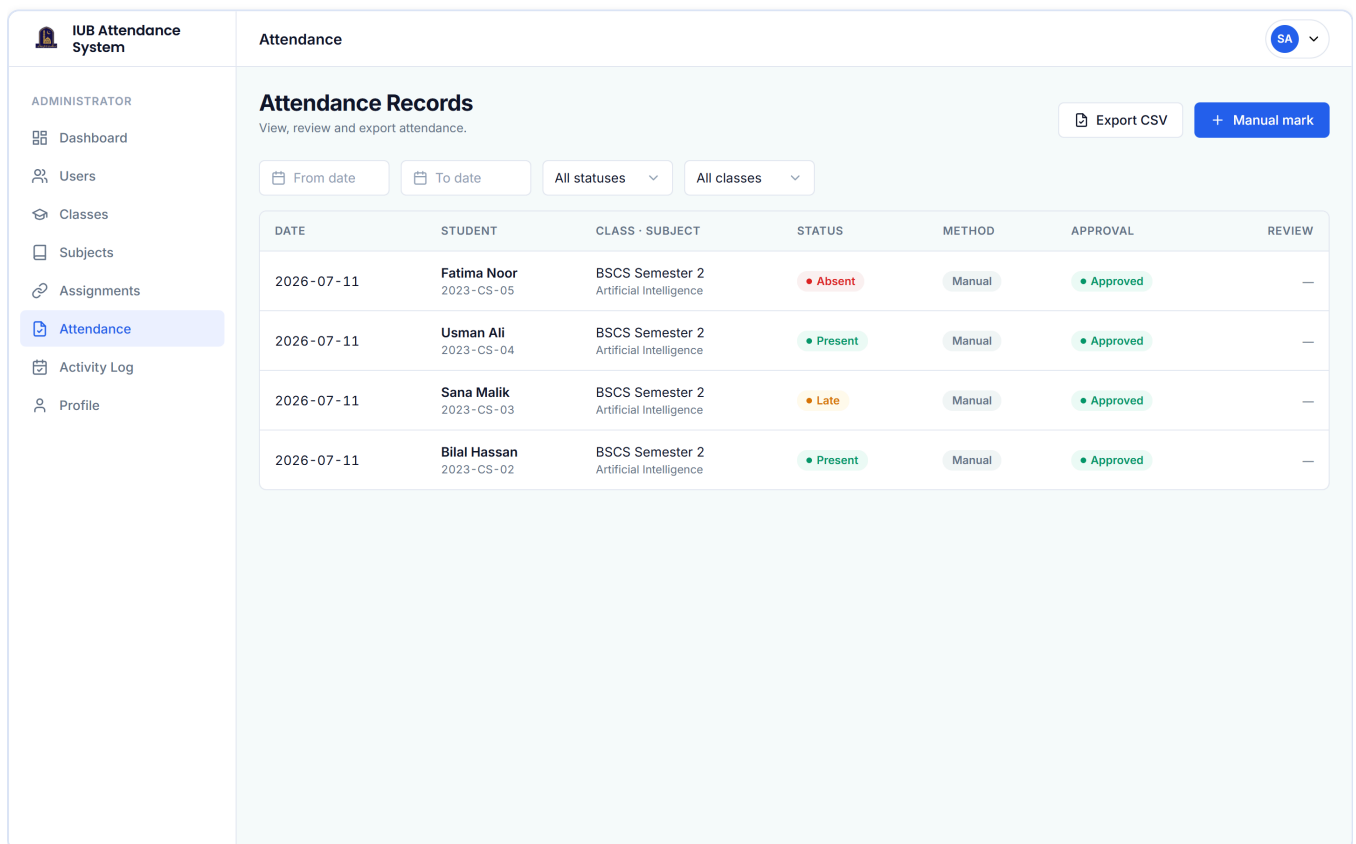


Fig 11.5 — Attendance records & reports with filters, CSV export, and approve/reject actions.

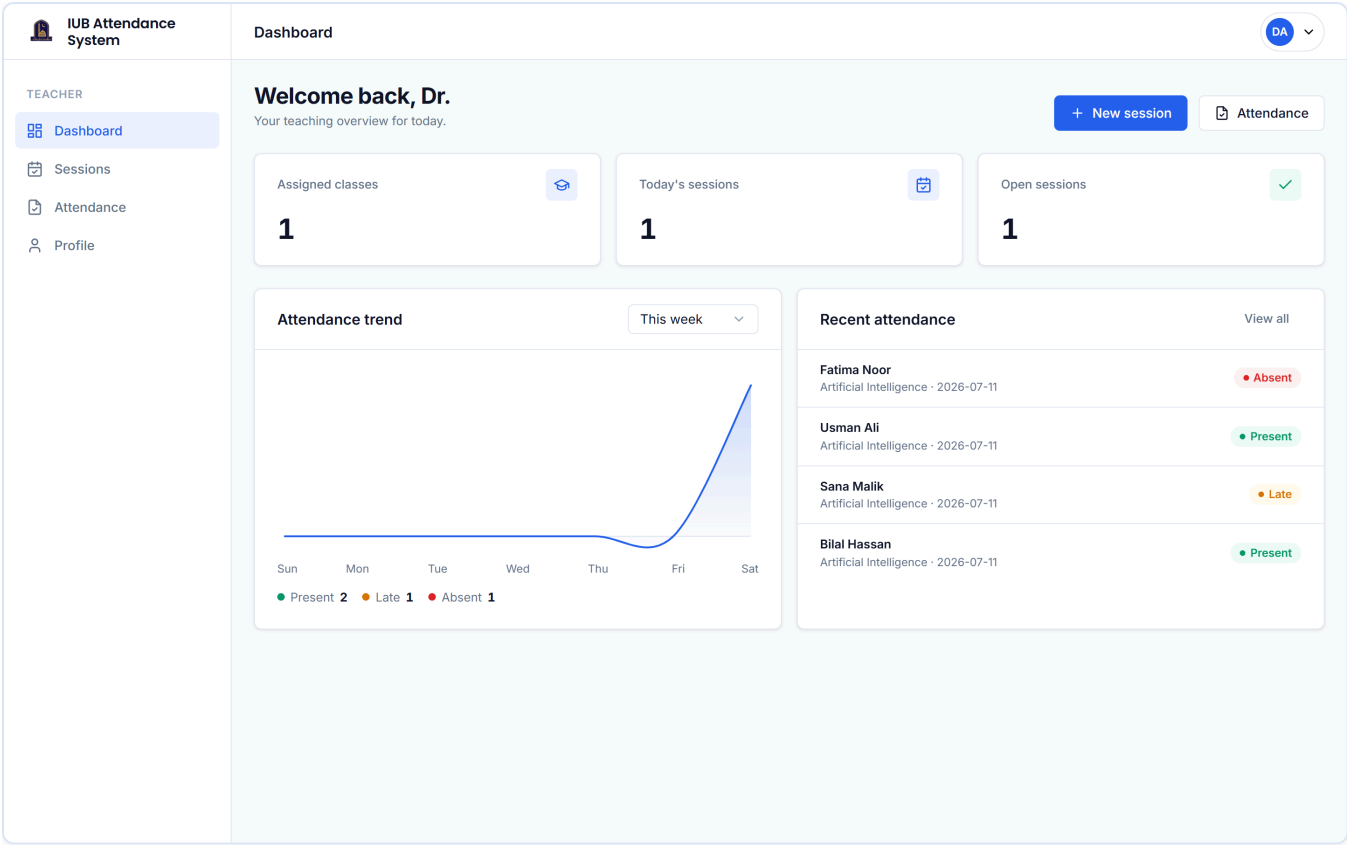


Fig 11.6 — Teacher dashboard with quick actions and attendance trend.

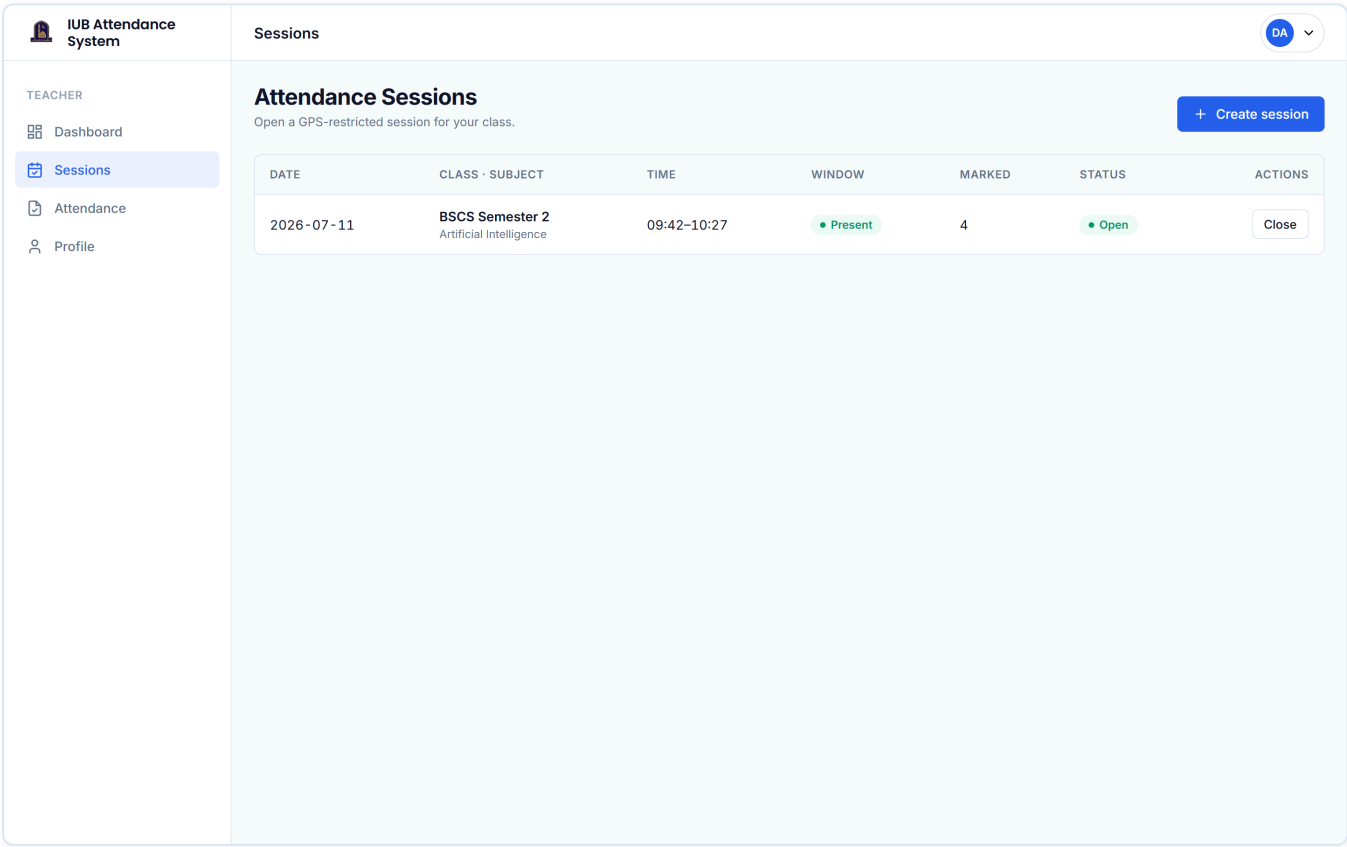


Fig 11.7 — Teacher's attendance sessions list (create, view window state, close).

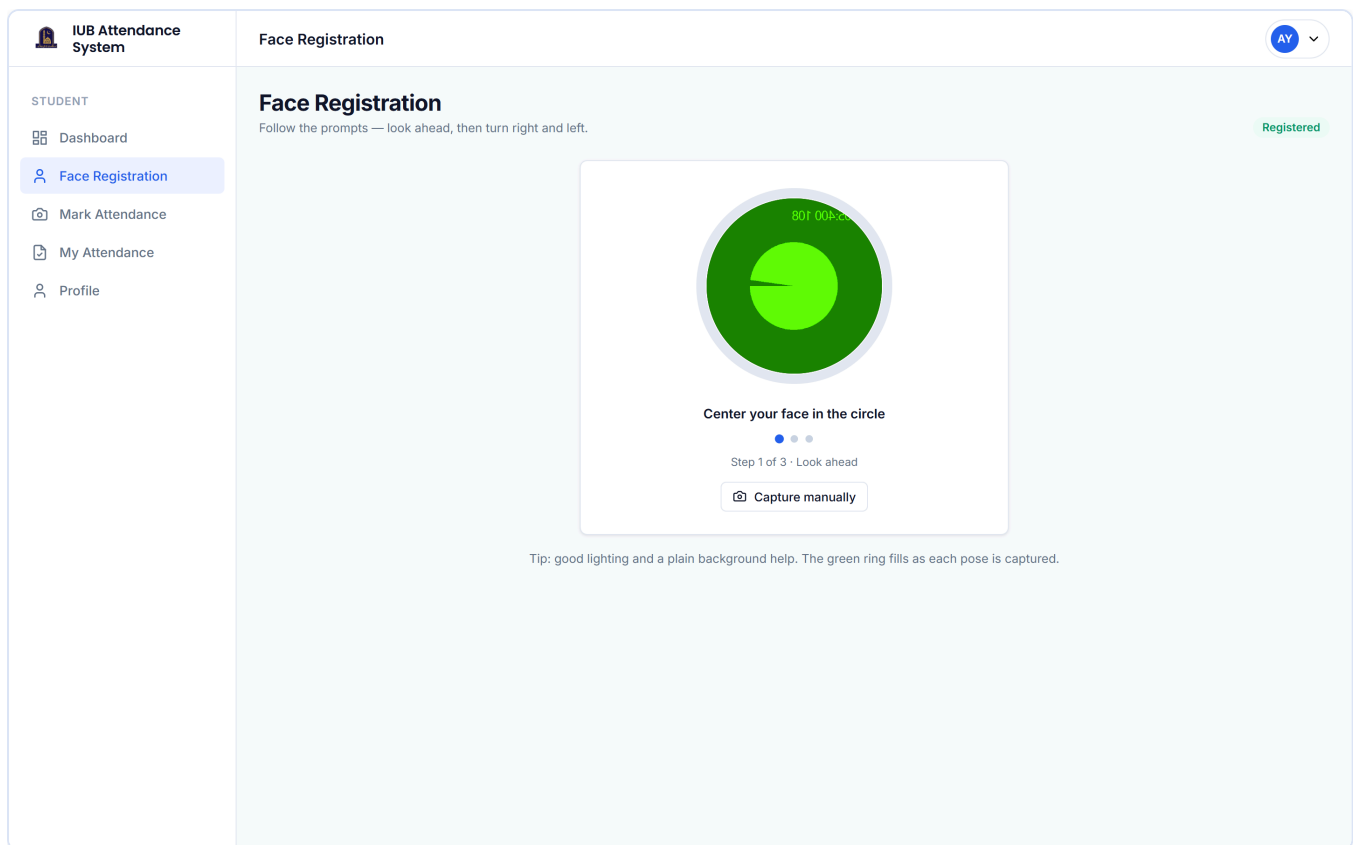


Fig 11.8 — Student face registration: guided, auto-capturing camera with a live progress ring.

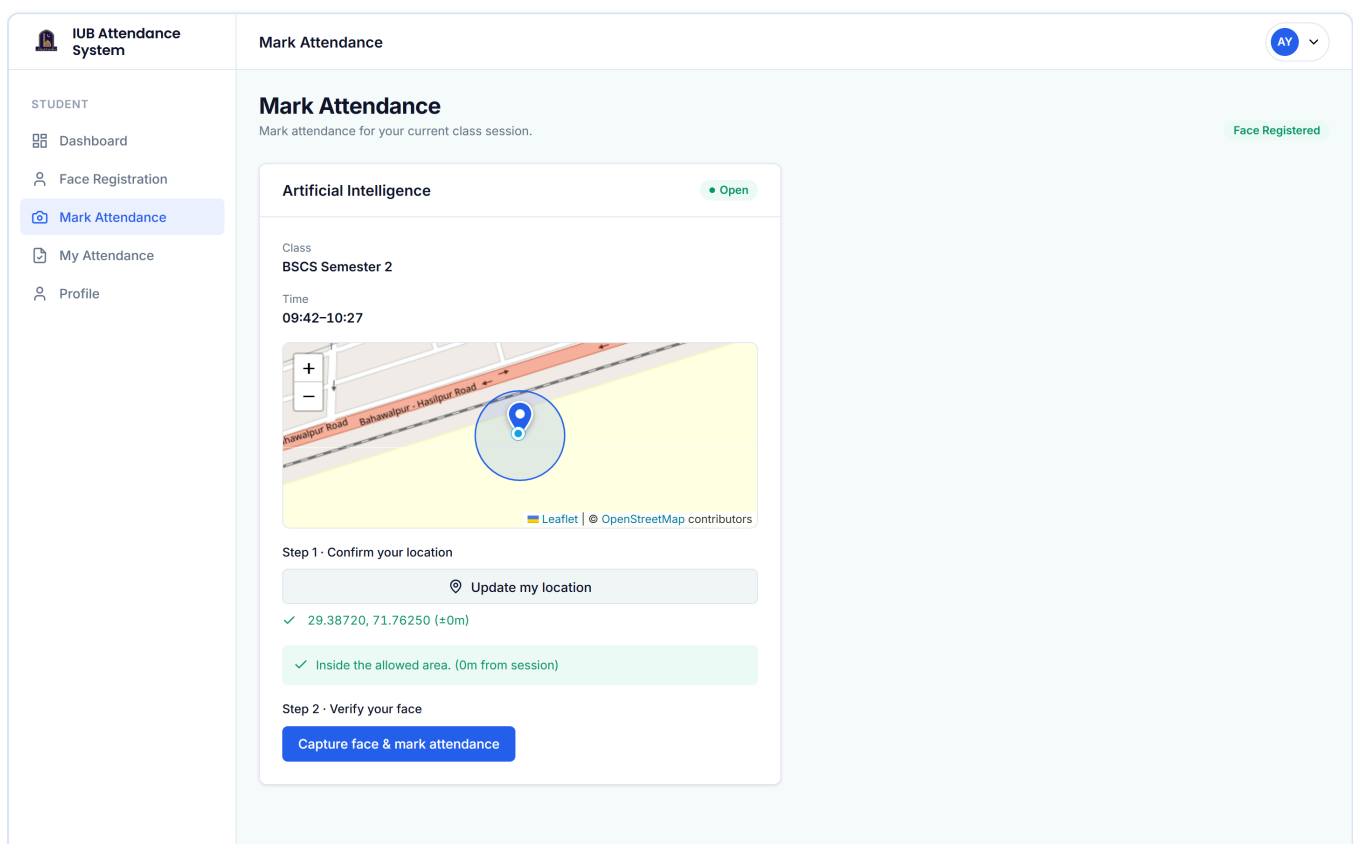


Fig 11.9 — Student Mark Attendance: session details, live map with the allowed radius and the student's location, and the two-step (location + face) flow.

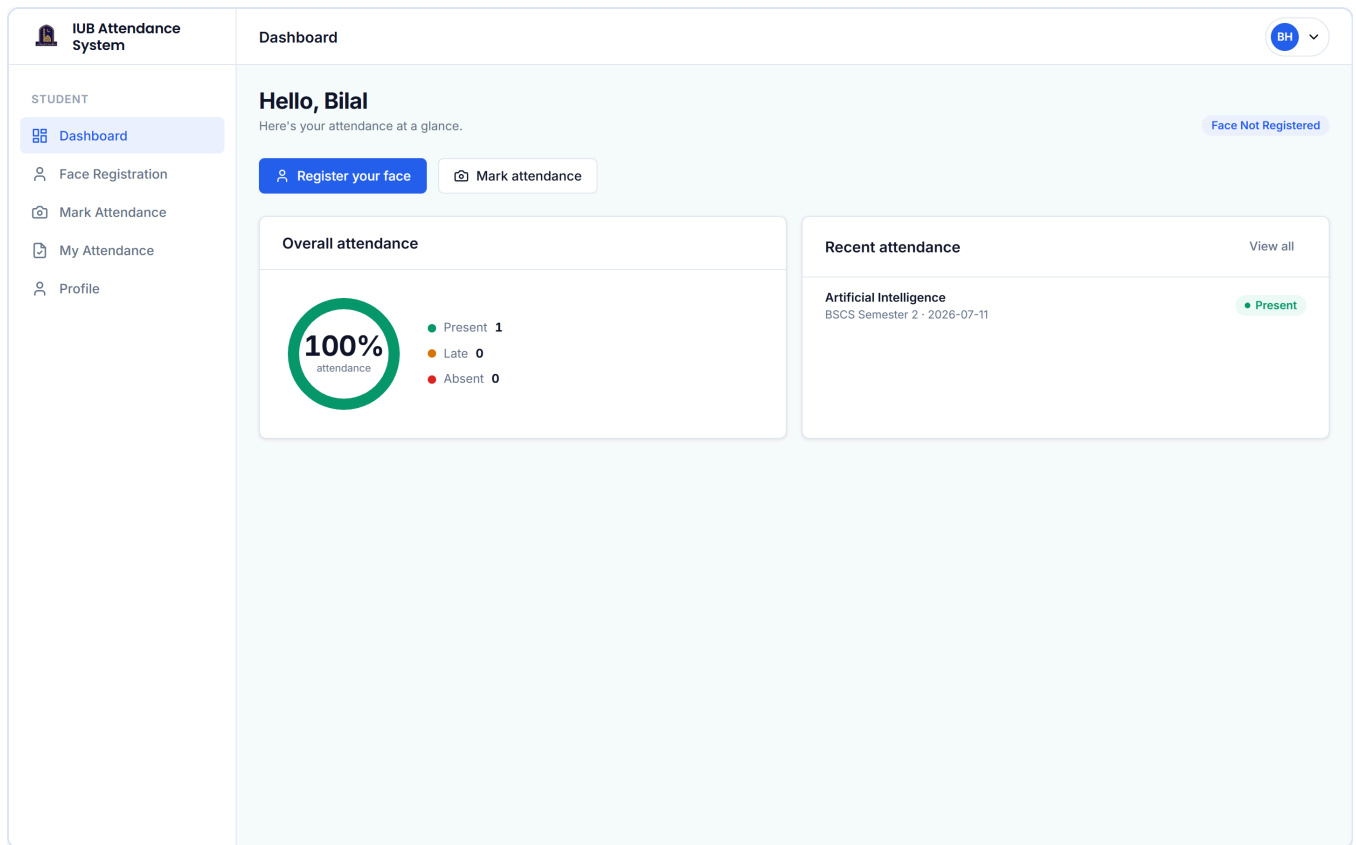


Fig 11.10 — Student dashboard with attendance-percentage ring and recent attendance.

11.1 Mobile (Progressive Web App)

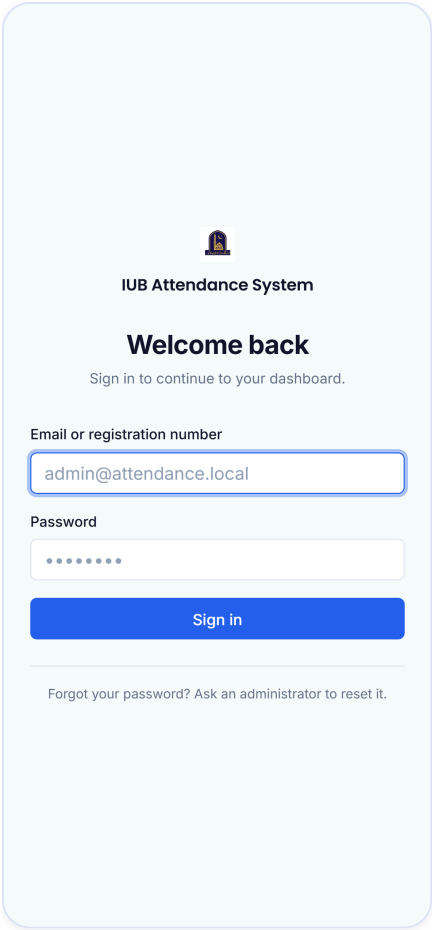


Fig 11.11 — Login (mobile)

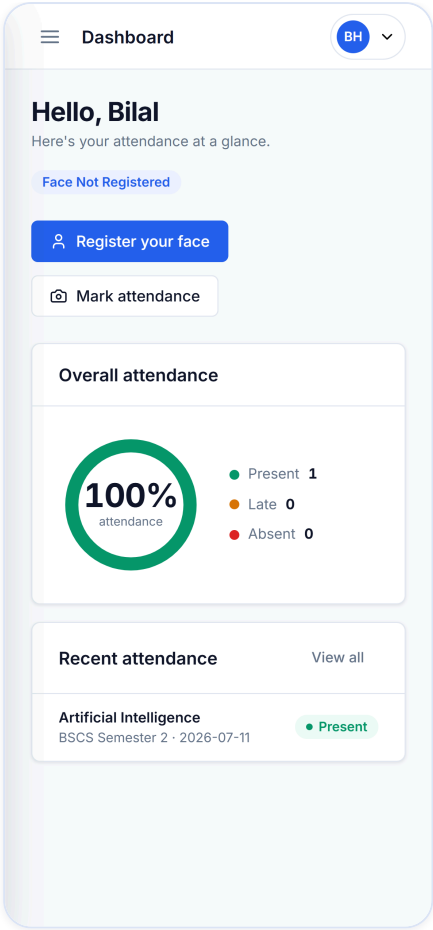


Fig 11.12 — Dashboard (mobile)

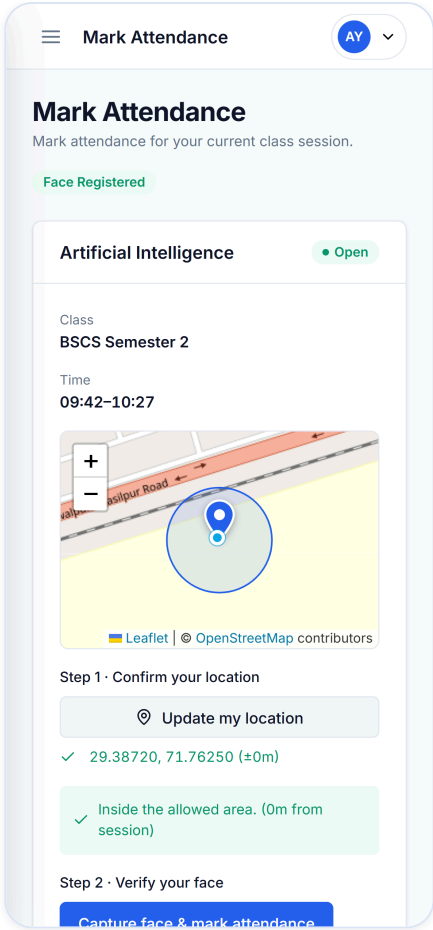


Fig 11.13 — Mark Attendance (mobile)

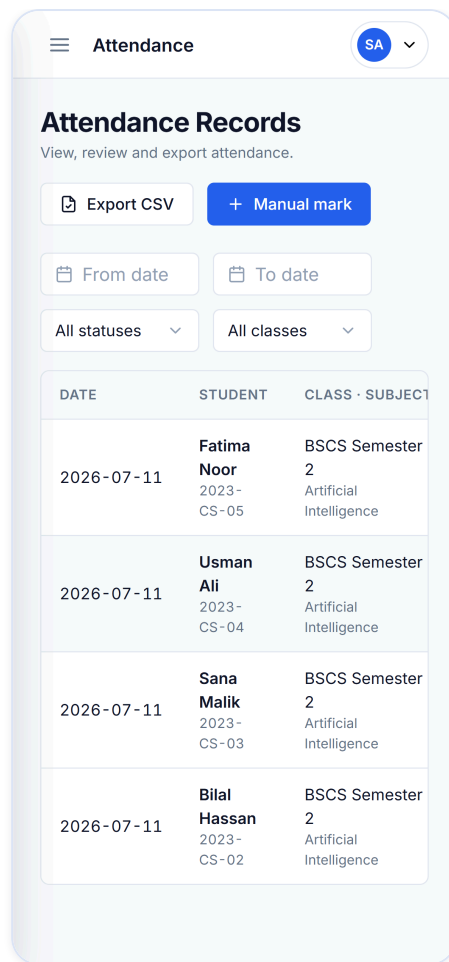


Fig 11.14 — Attendance records (mobile)

12. Security

- Passwords stored using **bcrypt** hashing; never in plain text.
- Stateless **JWT** authentication with a server-side secret and expiry.
- **Role-based authorization** on every protected API route.
- Inactive users cannot log in; **deactivating a student disables their face data** so they can no longer be recognised.
- Identity is verified **against the logged-in account only**, preventing proxy attendance.
- Database credentials and secrets are kept in **environment variables**, never in code or the repository.
- All traffic is served over **HTTPS** (also required by browsers for camera and GPS access), with CORS restricted to the frontend origin.

13. Deployment

The system is deployed on a fully free, production-grade stack:

Component	Platform	Notes
Database	Neon PostgreSQL	Serverless free tier; tables auto-created on first start.
Backend API	Render (Docker)	FastAPI + ONNX engine in a container; HTTPS; scales to zero when idle.
Frontend	Vercel	Static React build served globally over HTTPS.
Source control	GitHub	Single repository; automatic redeploys on push.

Because the AI engine was optimised to run on CPU within 512 MB, the entire backend fits the free hosting tier — no paid compute is required. The frontend is an installable PWA, so students can add it to their home screen and use it like a native app.

14. Testing & Verification

Each part of the system was verified against a live database before release:

Area	What was verified
Authentication	Login by email/registration number, case-insensitive email, role guards, password reset, deactivation blocks login.
Management	Create/edit/delete of users, classes, subjects, and teacher assignments with validation.
Sessions	Correct present/late/closed window states; one-active-window enforcement; automatic closing of expired sessions.
GPS	Haversine accuracy; acceptance inside the radius; rejection outside; accuracy-limit handling.
Face recognition	Same-person acceptance (0.91 confidence) and different-person rejection on real images.
Attendance flow	Full pipeline, duplicate prevention, manual marking, and approve→present/late, reject→absent.

15. Conclusion & Future Enhancements

15.1 Conclusion

The AI-Powered Web-Based Smart Attendance System successfully demonstrates that reliable, tamper-resistant attendance can be achieved using only students' own smartphones and free, open-source software. By requiring both a matching face and a valid in-radius location within a teacher-controlled session, it eliminates proxy attendance and manual registers while remaining completely free to run. The result is a complete, deployed, mobile-installable application covering administrators, teachers, and students end to end.

15.2 Future Enhancements

- **Liveness / anti-spoofing** (blink or head-movement checks) to defeat photo attacks.
- **QR code or campus Wi-Fi verification** combined with GPS for stronger presence proof.
- **Email-based password reset** and notifications.
- **PDF reports** and richer analytics.
- **Native mobile app** and offline attendance queueing.

— End of Document —